

Колледж Университета «Сириус»

ОТЧЁТ

по лабораторной работе № 1

ИНДЕКСЫ И EXPLAIN ANALYZE В POSTGRESQL

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

подпись, дата

У. И. Усачева

Руководитель:

преподаватель

подпись, дата

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Работа с тестовой базой данных	6
1.1 Задания 1–2. Таблица orders и данные	6
1.2 Задание 3. EXPLAIN	6
1.3 Задание 4. EXPLAIN ANALYZE	6
1.4 Задание 5. Sequential Scan	8
1.5 Задание 6. Первый B-tree индекс	9
1.6 Задание 7. Индекс и низкая селективность	9
1.7 Задание 8. Селективность	10
1.8 Задание 9. Range Query	11
1.9 Задание 10. Bitmap Scan	12
1.10 Задание 11. Несколько условий и несколько индексов	13
1.11 Задание 12. Composite Index	14
1.12 Задание 13. Порядок колонок	15
1.13 Задание 14. WHERE + ORDER BY	16
1.14 Задание 15. Pagination Query — GET /users/{user_id}/orders	18
1.15 Задание 16. Index Only Scan	19
1.16 Задание 17. Partial Index	21
1.17 Задание 18. Expression Index	22
1.18 Задание 19. Цена индексов на запись	23
1.19 Задание 20. Неиспользуемые индексы	23
1.20 Задание 21. Оптимизация запроса	25
2 Оптимизация собственного сервиса TaskManager	27
2.1 Задание 22. Scaling Entity	27
2.2 Задание 23. Реальные запросы	28
2.3 Задание 24. Базовое измерение	29
2.4 Задание 25. Анализ текущих индексов	30
2.5 Задание 26. Потенциальные проблемы	32
2.6 Задание 27. Выбранный запрос и инженерный процесс	33

2.7 Задание 28. Изменение в проекте	35
2.8 Задание 29. Повторное измерение.....	36
3 Почему нельзя индексировать всё (задание 30).....	39
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	43
ПРИЛОЖЕНИЕ А Миграция с индексами для GET /api/tasks.....	44
ПРИЛОЖЕНИЕ Б Миграция удаления дублирующих индексов	45

ВВЕДЕНИЕ

Лабораторная работа посвящена анализу выполнения SQL-запросов в PostgreSQL и выбору индексов на основе измерений. Главное правило работы: запрос нельзя считать оптимизированным только потому, что создан индекс, — любое изменение подтверждается сравнением `EXPLAIN ANALYZE` до и после.

Цель работы — научиться анализировать планы выполнения запросов с помощью `EXPLAIN` и `EXPLAIN ANALYZE`, выбирать индексы под конкретные запросы и применить эти знания для оптимизации собственного сервиса.

Для достижения цели решались следующие задачи:

- изучить основные способы доступа к данным: Seq Scan, Index Scan, Bitmap Heap Scan, Index Only Scan;
- исследовать влияние селективности условия на выбор плана выполнения;
- сравнить одиночные, составные, частичные, покрывающие индексы и индексы по выражению;
- оценить цену индексов при операциях записи;
- найти проблемные запросы собственного сервиса TaskManager, оптимизировать их индексами, добавленными через миграции, и подтвердить результат измерениями.

Работа состоит из двух частей. В первой части эксперименты проводятся на тестовой таблице `orders` (1 000 000 строк), во второй — на базе данных сервиса TaskManager (REST API на .NET 8, PostgreSQL 16), заполненной 1 000 000 задач. При выполнении работы использовались документация PostgreSQL [1–4], документация Liquibase [5] и методические указания к лабораторной работе [6].

Таблица 1 – Среда выполнения экспериментов

Параметр	Значение
СУБД	PostgreSQL 16.14 (образ <code>postgres:16-alpine</code>) в Docker Desktop, Windows 11

Параметр	Значение
shared_buffers	128 МБ
work_mem	4 МБ
random_page_cost / seq_page_cost	4 / 1 (значения по умолчанию)
max_parallel_workers_per_gather	2

Все эксперименты автоматизированы и повторяются одной командой:

Таблица 2 – Скрипты экспериментов и файлы результатов

Часть	Скрипты	Сырые результаты EXPLAIN ANALYZE
Часть 1 (тестовая БД lab01)	scripts/lab-01/01-setup.sql ... 11-final.sql, запуск bash scripts/lab-01/run-all.sh	docs/lab-01/results/01-*.txt ... 11-*.txt
Часть 2 (БД сервиса taskmanager_db)	scripts/generate-data.sql, scripts/lab-01/part2-capture-sql.sh, scripts/lab-01/part2-explain.sql	docs/lab-01/results/part2-*.txt

Правила измерений:

– Время указано по одному прогону на «тёплом» кэше (данные уже в памяти), поэтому абсолютные цифры зависят от машины. Сравнивать нужно порядок величин и планы.

– Различия меньше ~0,1 мс — это шум. В таких случаях сравнивается Buffers (сколько страниц по 8 КБ прочитано): эта метрика от нагрузки на машину не зависит.

– Скрипты Части 1 выполняются строго по порядку: набор индексов переходит из задания в задание, и в начале каждого шага он явно приводится к нужному состоянию.

1 Работа с тестовой базой данных

1.1 Задания 1–2. Таблица orders и данные

Таблица создана по заданию, 1 000 000 строк сгенерированы запросом из методички за 3,6 с.

Размер таблицы — 85 МБ (10 826 страниц), первичный ключ — 21 МБ. После загрузки выполнен `ANALYZE orders`.

1.2 Задание 3. EXPLAIN

Листинг 1 – План выполнения (задание 3)

```
Gather (cost=1000.00..17035.43 rows=11 width=53)
  Workers Planned: 2
  -> Parallel Seq Scan on orders (cost=0.00..16034.33 rows=5 width=53)
      Filter: (user_id = 123)
```

1) **Какой план выбран?** Gather над Parallel Seq Scan: таблицу параллельно читают два фоновых процесса и ведущий процесс, а Gather собирает их результаты.

2) **Тип сканирования** — последовательное (Sequential Scan), в параллельном варианте.

3) **Как PostgreSQL ищет строки?** Индекса по `user_id` нет, поэтому единственный способ — прочитать все страницы таблицы и проверить `Filter: user_id = 123` для каждой строки. Оценка `rows=11` взята из статистики `pg_stats`: около 1 млн строк на ~90 тыс. различных `user_id`. `cost` — условные единицы планировщика: чтение страницы подряд стоит 1 (`seq_page_cost`), плюс небольшая плата за обработку каждой строки. Параллельный план выбран, потому что таблица больше `min_parallel_table_scan_size` (8 МБ).

1.3 Задание 4. EXPLAIN ANALYZE

Листинг 2 – План выполнения (задание 4)

```
Gather (cost=1000.00..17035.43 rows=11 width=53) (actual time=3.435..31.715 rows=15 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  -> Parallel Seq Scan on orders (cost=0.00..16034.33 rows=5 width=53) (actual
```

```

time=5.024..16.519 rows=5 loops=3)
  Filter: (user_id = 123)
  Rows Removed by Filter: 333328
Planning Time: 0.042 ms
Execution Time: 31.739 ms

```

Таблица 3 – Задание 4. EXPLAIN ANALYZE

Что искали	Значение
Тип сканирования	Parallel Seq Scan
Planning Time	0,042 мс
Execution Time	31,7 мс
actual rows	15 (в узле сканирования rows=5 loops=3: 5 строк × 3 процесса)
Rows Removed by Filter	333 328 на процесс, то есть ~1 000 000 проверенных и отброшенных строк
Buffers (прогон с BUFFERS)	shared hit=10826 — прочитана вся таблица

1) **Чем EXPLAIN отличается от EXPLAIN ANALYZE?** EXPLAIN только строит план и показывает оценки (cost, rows), сам запрос не выполняется. EXPLAIN ANALYZE выполняет запрос по-настоящему и добавляет фактические значения: actual time, actual rows, loops, Execution Time. Поэтому EXPLAIN ANALYZE для INSERT/UPDATE/DELETE реально меняет данные, и оборачивать его нужно в BEGIN; ... ROLLBACK;.

2) **Почему EXPLAIN ANALYZE дольше?** EXPLAIN занял 1,1 мс (только планирование), EXPLAIN ANALYZE — 32 мс. Во втором случае запрос действительно прочитал 10 826 страниц и проверил миллион строк. К этому добавляются накладные расходы на замер времени каждого узла: SELECT * FROM orders с таймингом шёл 86,8 мс, а с EXPLAIN (ANALYZE, TIMING OFF) — 40,2 мс. В обратную сторону тоже есть поправка: EXPLAIN ANALYZE не передаёт строки клиенту, поэтому реальный запрос с сетью может идти дольше.

3) **Estimated и actual rows.** Estimated — прогноз по статистике, которую ANALYZE собирает по выборке строк (для default_statistics_target=100 это 30 000 строк). Actual — факт выполнения. Здесь 11 против 15, прогноз хороший. Если прогноз расходится с фактом на порядки, планировщик выбирает плохой план. Лечится это через ANALYZE, увеличение statistics target или расширенную статистику

(CREATE STATISTICS). Важно: actual rows в узле указывается на один проход, общее число равно rows × loops.

1.4 Задание 5. Sequential Scan

Таблица 4 – Задание 5. Sequential Scan

Запрос	План	Строк	Execution Time
SELECT * FROM orders	Seq Scan	1 000 000	86,8 мс
WHERE amount > 0	Seq Scan, Rows Removed by Filter: 1	999 999	144,3 мс

1) **Тип сканирования** — Seq Scan (даже не параллельный: передавать миллион строк от рабочих процессов ведущему дороже, чем читать таблицу одним процессом).

2) **Почему Seq Scan?** Условие amount > 0 истинно для 99,99 % строк, а индекса по amount нет. Но даже с индексом чтение всей таблицы через него было бы дороже: пришлось бы прочитать ещё и сам индекс, а к страницам таблицы обращаться вразнобой.

3) **Всегда ли Seq Scan — плохо?** Нет. Это оптимальный план, когда нужна большая часть таблицы.

Вывод. Последовательное сканирование эффективнее индекса, когда:

- запрос возвращает заметную долю таблицы. В экспериментах ниже граница оказалась между 20 % и 50 %: status 20 % ещё идёт через индекс, 75 % — уже Seq Scan; created_at за месяц (4 %) — через индекс, за год (50 %) — Seq Scan;
- таблица маленькая (несколько страниц);
- нет подходящего индекса или выражение в WHERE не совпадает с индексом (задание 18);
- считается агрегат по всей таблице.

Последовательное чтение дешевле случайного (seq_page_cost=1 против random_page_cost=4) и хорошо распараллеливается.

1.5 Задание 6. Первый B-tree индекс

Листинг 3 – SQL-запрос (задание 6)

```
CREATE INDEX idx_orders_user_id ON orders(user_id); -- 0.37 s, 9 MB
```

Листинг 4 – План выполнения (задание 6)

```
Bitmap Heap Scan on orders (cost=4.51..47.60 rows=11 width=53) (actual time=0.033..0.055
rows=15 loops=1)
  Recheck Cond: (user_id = 123)
  Heap Blocks: exact=15
  Buffers: shared hit=18
  -> Bitmap Index Scan on idx_orders_user_id (actual time=0.022..0.022 rows=15 loops=1)
        Index Cond: (user_id = 123)
        Buffers: shared hit=3
Execution Time: 0.124 ms
```

Таблица 5 – Задание 6. Первый B-tree индекс

Метрика	До индекса	После индекса
Тип Scan	Parallel Seq Scan	Bitmap Index Scan → Bitmap Heap Scan
Execution Time	32,7 мс	0,27 мс (повторно 0,12 мс)
Количество обработанных строк	~1 000 000 (15 найдено, остальные отброшены фильтром), 10 826 страниц	15 строк, 18 страниц (3 индексных + 15 табличных)
Используемый индекс	нет	idx_orders_user_id

1) **План изменился:** вместо полного прохода по таблице — поиск по индексу.

2) **Индекс используется:** Index Cond: (user_id = 123).

3) **Время сократилось** с 32,7 до 0,12–0,27 мс, то есть в 120–260 раз. Страниц прочитано в 600 раз меньше.

Вместо обычного Index Scan выбран Bitmap: 15 строк лежат на 15 разных страницах, и планировщик оценил чтение страниц в физическом порядке чуть дешевле. Для такого количества строк разница несущественна.

1.6 Задание 7. Индекс и низкая селективность

Индекс idx_orders_status. Генератор из задания 2 распределяет статусы **равномерно**, по ~25 %:

Таблица 6 – Задание 7. Индекс и низкая селективность

status	План	Heap Blocks	Execution Time
PAID	Bitmap Heap Scan	10 826 (вся)	48,0 мс

status	План	Heap Blocks	Execution Time
	(idx orders status)	таблица)	
NEW	Bitmap Heap Scan	10 826	49,0 мс
DELIVERED	Bitmap Heap Scan	10 826	47,6 мс
CANCELLED	Bitmap Heap Scan	10 826	49,2 мс
PAID, индексы запрещены (для сравнения)	Seq Scan	—	78,0 мс

При 25 % bitmap всё равно читает **все** страницы таблицы: строки с каждым статусом разбросаны по всему файлу. Выигрыш перед Seq Scan (48 против 78 мс) даёт лишь то, что bitmap проверяет не миллион строк, а только 250 тысяч отмеченных. Это верхняя граница, до которой индекс ещё полезен.

1) **Во всех ли случаях используется индекс?** При равномерном распределении — да, Bitmap для всех четырёх значений. После перекоса данных (задание 8) для DELIVERED (75 %) планировщик выбирает Seq Scan, хотя индекс есть.

2) **Почему Seq Scan вместо индекса?** Планировщик сравнивает стоимость. Если условие выбирает большую долю строк, доступ через индекс — это чтение индекса плюс случайные обращения почти к каждой странице таблицы, что дороже одного последовательного прохода.

3) **Как количество строк влияет на план?** Стоимость доступа через индекс растёт пропорционально числу строк, а стоимость Seq Scan постоянна и зависит только от размера таблицы. Поэтому с ростом результата план переключается: Index Scan (единицы строк) → Bitmap Scan (проценты) → Seq Scan (десятки процентов).

1.7 Задание 8. Селективность

Листинг 5 – Равномерное распределение и то, что знает о нём планировщик

status	cnt	pct	most_common_vals	most_common_freqs
CANCELLED	249346	24.93	{PAID, DELIVERED, NEW, CANCELLED}	{0.2536, 0.2501, 0.2492, 0.2471}
DELIVERED	249462	24.95		
NEW	250304	25.03		
PAID	250888	25.09		

Реальные статусы редко распределены поровну, поэтому status был пересчитан в реалистичную пропорцию (NEW 1 %, CANCELLED 4 %, PAID

20 %, DELIVERED 75 %). Затем выполнены VACUUM FULL и ANALYZE, чтобы убрать миллион мёртвых версий строк после UPDATE:

Таблица 7 – Задание 8. Селективность

status	Строк	% таблицы	Используется индекс	Тип сканирования	Heap Blocks	Execution Time
NEW	9 946	0,99 %	да	Bitmap Heap Scan	6 552	19,2 мс
CANCELLED	39 793	3,98 %	да	Bitmap Heap Scan	10 823	24,7 мс
PAID	199 289	19,93 %	да	Bitmap Heap Scan	11 109	39,3 мс
DELIVERED	750 972	75,10 %	нет	Seq Scan	—	107,7 мс

Самое интересное здесь — столбец Heap Blocks. Даже 1 % строк (NEW) лежит на 6 552 страницах из ~11 000, то есть на 60 % таблицы, потому что строки разбросаны случайно. Для эффективности индекса важна не только доля строк, но и то, сколько страниц нужно ради них прочитать.

Вывод. Чем селективнее условие (чем меньшую долю таблицы оно выбирает), тем полезнее индекс. На колонке с 4 значениями отдельный индекс помогает только для редких значений. Для таких колонок лучше подходят частичный индекс (задание 17) или составной индекс, где низкоселективная колонка стоит не первой.

1.8 Задание 9. Range Query

Таблица 8 – Задание 9. Range Query

Диапазон	Строк	%	План	Execution Time
7 дней, без индекса	9 606	0,96 %	Parallel Seq Scan	62,2 мс
1 день	1 327	0,13 %	Bitmap Heap Scan (idx_orders_created_at)	1,8 мс
7 дней	9 606	0,96 %	Bitmap Heap Scan	8,3 мс
1 месяц	41 484	4,1 %	Bitmap Heap Scan	19,4 мс
1 год	500 532	50,1 %	Seq Scan	177,2 мс

1) **Во всех ли случаях используется индекс?** Нет: для диапазона в год — Seq Scan.

2) **Как размер диапазона влияет на план?** Узкий диапазон — мало строк, и индекс ускоряет запрос в 7–35 раз. С расширением диапазона стоимость доступа через индекс растёт, и начиная с некоторой доли таблицы планировщик переходит на Seq Scan.

3) **Когда индекс становится невыгоден?** Когда диапазон покрывает большую часть таблицы (здесь 50 %). Ещё один фактор — физический порядок строк. В тестовых данных `created_at` сгенерирован случайно и не связан с порядком строк на диске (`pg_stats.correlation ≈ 0`), поэтому даже 4 % строк размазаны по всем страницам. В реальной таблице, куда строки пишутся по времени, корреляция близка к 1, и индекс остаётся выгодным на гораздо более широких диапазонах.

1.9 Задание 10. Bitmap Scan

Листинг 6 – План выполнения (задание 10)

```
Bitmap Heap Scan on orders (actual time=4.339..19.095 rows=39793 loops=1)
  Recheck Cond: ((status)::text = 'CANCELLED'::text)
  Heap Blocks: exact=10823
-> Bitmap Index Scan on idx_orders_status (actual time=2.855..2.856 rows=39793 loops=1)
    Index Cond: ((status)::text = 'CANCELLED'::text)
Execution Time: 20.441 ms
```

Такой же план получился для `created_at > NOW() - INTERVAL '1 month'` (41 484 строки, 18,5 мс). Для `amount BETWEEN 1000 AND 3000` индекса нет, поэтому Seq Scan (117 мс).

Таблица 9 – Сравнение способов доступа для `status = 'CANCELLED'` (4 %)

Настройка	План	Buffers	Execution Time
по умолчанию	Bitmap Heap Scan	10 860	16,8–20,4 мс
<code>enable_bitmapscan = off</code>	Parallel Seq Scan (не Index Scan!)	—	32,8 мс
<code>enable_bitmapscan = off,</code> <code>enable_seqscan = off</code>	Index Scan	10 860	12,2 мс

Без bitmap планировщик предпочёл обычному Index Scan даже Seq Scan: для 40 тысяч строк он оценивает Index Scan в 29 075 единиц против 12 058 у Bitmap, потому что считает каждое обращение к таблице случайным чтением с диска (`random_page_cost = 4`). Фактически все данные лежат в памяти, и принудительный Index Scan оказался даже быстрее. Этот же эффект ещё раз проявится в задании 14.

1) **Что делает Bitmap Index Scan?** Проходит по индексу и строит в памяти битовую карту адресов (TID) подходящих строк, упорядоченную по

номеру страницы. К таблице не обращается. Если карта не помещается в `work_mem`, она становится «lossy»: хранит только номера страниц.

2) **Что делает Bitmap Heap Scan?** Читает страницы таблицы в физическом порядке по карте, каждую страницу ровно один раз, и перепроверяет условие (`Recheck Cond`). Для lossy-страниц это обязательно, потому что внутри страницы неизвестно, какие именно строки подходят.

3) **Почему Bitmap вместо Index Scan?** `Index Scan` обращается к таблице в порядке индекса, то есть прыжками по файлу, и одну страницу может прочитать много раз. `Bitmap` сортирует обращения, приближая их к последовательному чтению, и умеет объединять несколько индексов (`BitmapAnd/BitmapOr`). Цена — теряется порядок индекса (для `ORDER BY` нужен `Sort`), и нельзя остановиться раньше при `LIMIT`. Поэтому `Bitmap` хорош для «средних» выборок: от сотен до десятков тысяч строк.

1.10 Задание 11. Несколько условий и несколько индексов

Листинг 7 – Запрос из задания при индексах (`user_id`), (`status`), (`created_at`)

```
Bitmap Heap Scan on orders (actual time=0.057..0.079 rows=4 loops=1)
  Recheck Cond: (user_id = 123)
  Filter: ((status)::text = 'PAID'::text)
  Rows Removed by Filter: 11
-> Bitmap Index Scan on idx_orders_user_id (actual rows=15 loops=1)
    Index Cond: (user_id = 123)
Execution Time: 0.114 ms
```

1) **Один индекс или несколько?** Один — `idx_orders_user_id`. Статус проверяется фильтром на уже найденных 15 строках.

2) **Есть ли BitmapAnd?** В этом запросе нет. `user_id` уже сужает выборку до 15 строк, а чтобы построить карту по `status = 'PAID'`, пришлось бы прочитать ~200 000 записей индекса. Проверить статус у 15 строк гораздо дешевле.

Чтобы увидеть `BitmapAnd`, нужны два условия, каждое из которых по отдельности выбирает много строк, а вместе — мало:

Листинг 8 – План выполнения (задание 11)

```
-- status = 'NEW' (1%) AND created_at за месяц (4%) → 410 строк
Bitmap Heap Scan on orders (actual time=5.426..6.024 rows=410 loops=1)
```

```

Heap Blocks: exact=398
-> BitmapAnd
    -> Bitmap Index Scan on idx_orders_status      (actual rows=9946)
    -> Bitmap Index Scan on idx_orders_created_at  (actual rows=41484)
Execution Time: 6.195 ms

```

Пересечение карт сократило чтение таблицы с 6 552 страниц (только по NEW) до 398. Для `user_id = 123 OR user_id = 456` получился `BitmapOr` (28 строк, 0,15 мс). В задании 21 до оптимизации `BitmapAnd` объединяет сразу три индекса.

3) Почему PostgreSQL может объединять индексы? Результат `Bitmap Index Scan` — это множество адресов строк, а пересечение или объединение битовых карт стоит дёшево. Это позволяет обслуживать разные комбинации условий набором одиночных индексов, не создавая составной индекс под каждую комбинацию. Цена: каждый индекс нужно прочитать по его условию целиком, и остаётся обращение к таблице.

1.11 Задание 12. Composite Index

У пользователя 123 всего 15 заказов, поэтому разница между планами тонет в шуме. Для наглядности добавлен «крупный» пользователь 100001 с 50 000 заказов (4,8 % таблицы, распределение статусов то же): так ведёт себя активный клиент в реальном сервисе.

Таблица 10 – Задание 12. Composite Index

	user 123: план	время / Buffers	user 100001: план	время / Buffers
До: (user_id) + (status)	Index Scan idx_orders_user_id+ Filter (отброшено 11)	0,077 мс / 21	Bitmap по idx_orders_user_id+ Filter (отброшено 39 982)	5,95 мс / 598
После: (user_id, status)	Index Scan idx_orders_user_status, оба условия в Index Cond	0,074 мс / 7	Bitmap по idx_orders_user_status, фильтра нет	3,17 мс / 567

Размеры: `idx_orders_user_id` — 9,4 МБ, `idx_orders_status` — 7,2 МБ, `idx_orders_user_status` — 15 МБ.

1) Что эффективнее? Составной индекс: оба условия проверяются внутри индекса, ничего не читается «впустую». Для крупного пользователя

запрос стал в 1,9 раза быстрее, для обычного время одинаковое в пределах шума, но страниц читается в 3 раза меньше.

2) **Всегда ли составной индекс лучше?** Нет. Он больше по размеру и помогает только запросам, чьи условия совпадают с его ведущими колонками. Если первая колонка сама очень селективна (как `user_id`: ~10 строк на пользователя), вторая почти ничего не добавляет. Отдельные индексы универсальнее: их можно комбинировать через `BitmapAnd` в разных запросах. И каждый лишний индекс замедляет запись (задание 19).

3) **От чего зависит выбор?** От частоты и формы запросов (какие колонки встречаются вместе, равенство это или диапазон, есть ли `ORDER BY`), от селективности колонок и их комбинации, от остальных запросов (нужен ли кому-то отдельный индекс) и от нагрузки на запись. Составной индекс (а, b) обслуживает и запросы только по а (левый префикс), поэтому может заменить индекс (а).

1.12 Задание 13. Порядок колонок

Чтобы эффект был виден, проведён «чистый» эксперимент: удалены одиночные индексы по `user_id` и `created_at`, которые иначе подменяли бы исследуемый составной индекс.

Таблица 11 – Задание 13. Порядок колонок

Запрос	Только (<code>user_id</code> , <code>created_at</code>)	Только (<code>created_at</code> , <code>user_id</code>)
Q1: <code>user_id = 123</code>	Bitmap, Index Cond: <code>user_id = 123</code> — 0,14 мс	индекс неприменим → Parallel Seq Scan — 25,5 мс
Q2: <code>user_id = 123</code> AND <code>created_at > now() - 30d</code>	Index Scan, обе колонки в Index Cond — 0,057 мс	Index Scan, но просматривается весь 30-дневный диапазон, <code>user_id</code> проверяется внутри него — 1,9 мс
Q2 для пользователя 100001	прочитано 10 индексных страниц — 1,08 мс	прочитано 169 индексных страниц — 2,67 мс
Q3: <code>created_at > now() - 30d</code>	индекс неприменим → Parallel Seq Scan — 63,7 мс	Bitmap, Index Cond: <code>created_at > ...</code> — 19,3 мс

Когда созданы оба индекса, планировщик сам выбирает `(user_id, created_at)` для Q1 и Q2 и `(created_at, user_id)` для Q3.

1) **Как порядок колонок влияет на использование индекса?** B-tree отсортирован по первой колонке, а вторая упорядочена только внутри одинаковых значений первой. Поиск идёт по префиксу: условие на ведущую колонку превращается в узкий непрерывный диапазон. По одной второй колонке такого диапазона нет: PostgreSQL 16 пришлось бы читать весь индекс (skip scan появился только в PostgreSQL 18), поэтому планировщик выбирает Seq Scan.

2) **Почему это разные индексы?** У них разный порядок ключей. В `(user_id, created_at)` заказы одного пользователя лежат рядом и отсортированы по дате, поэтому «заказы пользователя за период» — один короткий отрезок. В `(created_at, user_id)` всё упорядочено по времени: «заказы за период» — один отрезок, а заказы конкретного пользователя разбросаны по всему индексу.

3) **Какой индекс для какого запроса:**

- Q1 → `(user_id, created_at)` (или просто `(user_id)`);
- Q2 → `(user_id, created_at)`;
- Q3 → `(created_at, ...)` или просто `(created_at)`.

Общее правило: колонки с условием равенства ставятся первыми, колонки с диапазоном или сортировкой — последними.

1.13 Задание 14. WHERE + ORDER BY

Таблица 12 – Задание 14. WHERE + ORDER BY

	user 123 (15 строк)	user 100001 (50 000 строк, без LIMIT)
До: <code>(user_id, created_at)</code>	Sort (quicksort, 26 кБ) ← Index Scan <code>idx_orders_user_id</code> — 0,134 мс	Sort (external merge, Disk 3,5 МБ) ← Bitmap Heap Scan — 30,8 мс
После: <code>(user_id, created_at DESC)</code>	Index Scan <code>idx_orders_user_created_at_desc</code> , Sort исчез — 0,084 мс	план не изменился: Sort + Bitmap — 33,1 мс

	user 123 (15 строк)	user 100001 (50 000 строк, без LIMIT)
После + random_page_cost = 1.1	—	Index Scan idx_orders_user_create d_at_desc, Sort исчез — 17,6 мс

Листинг 9 – План выполнения (задание 14)

```
-- после индекса, user 123
Index Scan using idx_orders_user_created_at_desc on orders (actual time=0.037..0.059 rows=15
loops=1)
  Index Cond: (user_id = 123)
Execution Time: 0.084 ms
```

1) **Исчезла ли операция Sort?** Для обычного пользователя — да. Для крупного без LIMIT — нет, и это наглядный пример того, что наличие индекса не гарантирует его использование. Чтобы отдать 50 000 строк в порядке индекса, нужно 50 000 обращений к таблице вразнобой. При random_page_cost = 4, рассчитанном на HDD, это оценивается дороже, чем Bitmap плюс сортировка. На самом деле данные в памяти, и при random_page_cost = 1.1 (типичная настройка для SSD) планировщик сам выбрал индекс, а запрос ускорился почти вдвое (17,6 против 33,1 мс). С enable_bitmapscan = off при настройках по умолчанию он предпочёл даже Parallel Seq Scan + Sort (42 мс). Вывод: параметры стоимости должны соответствовать реальному железу.

2) **Почему индекс может заменить сортировку?** B-tree хранит ключи упорядоченно. При фиксированном user_id записи уже идут по created_at DESC, поэтому, прочитав отрезок индекса, получаем строки сразу в нужном порядке. Индекс по возрастанию (user_id, created_at) тоже подходит: B-tree можно читать в обратную сторону (Index Scan Backward, 0,081 мс с LIMIT 20). Направление DESC в определении индекса принципиально только для смешанных сортировок, например ORDER BY a ASC, b DESC.

3) **Почему важен порядок колонок?** Порядок created_at совпадает с порядком индекса только внутри одного user_id, поэтому user_id (равенство) должен стоять первым. С индексом (created_at DESC,

user_id) планировщик его вообще не использовал: запрос пошёл через idx_orders_user_id + Sort.

1.14 Задание 15. Pagination Query — GET /users/{user_id}/orders

Листинг 10 – SQL-запрос (задание 15)

```
SELECT * FROM orders WHERE user_id = 123 ORDER BY created_at DESC LIMIT 20;
```

Листинг 11 – План до оптимизации (индексы (user_id), (created_at))

```
-- user 123
Limit (actual time=0.047..0.050 rows=15 loops=1)
  -> Sort (Sort Key: created_at DESC, quicksort 26kB)
      -> Index Scan using idx_orders_user_id on orders (actual rows=15 loops=1)
          Index Cond: (user_id = 123)
Execution Time: 0.079 ms          Buffers: 18

-- user 100001
Limit (actual time=0.172..0.536 rows=20 loops=1)
  -> Index Scan Backward using idx_orders_created_at on orders
      Filter: (user_id = 100001)
      Rows Removed by Filter: 396
Execution Time: 0.557 ms          Buffers: 421
```

Индекс: CREATE INDEX idx_orders_user_created_at_desc ON orders(user_id, created_at DESC);

План после:

Листинг 12 – План выполнения (задание 15)

```
Limit (actual time=0.035..0.062 rows=20 loops=1)
  -> Index Scan using idx_orders_user_created_at_desc on orders (actual rows=20 loops=1)
      Index Cond: (user_id = 100001)
Execution Time: 0.087 ms          Buffers: 23
```

Таблица 13 – Задание 15. Pagination Query — GET /users/{user_id}/orders

Метрика	До	После
user 123: план	Limit ← Sort ← Index Scan (idx_orders_user_id)	Limit ← Index Scan (idx_orders_user_created_at_desc)
user 123: время / Buffers	0,079 мс / 18	0,082 мс / 18
user 100001: план	Limit ← Index Scan Backward (idx_orders_created_at) + Filter	Limit ← Index Scan (idx_orders_user_created_at_desc)
user 100001: время / Buffers	0,557 мс / 421	0,087 мс / 23

Для крупного пользователя план «до» — удачное совпадение: он составляет 5 % таблицы, поэтому, идя по индексу `created_at` с конца, 20 его заказов находятся всего через 400 чужих. У обычного пользователя (0,001 % таблицы) такой план прошёл бы десятки тысяч записей, и планировщик разумно выбрал для него `idx_orders_user_id` + сортировку.

Почему такая структура индекса:

- `user_id` первым, потому что это условие равенства: поиск сразу попадает в отрезок одного пользователя;
- `created_at DESC` вторым: внутри отрезка строки уже упорядочены как в `ORDER BY`, и `Sort` не нужен;
- `LIMIT 20` останавливает чтение после 20 записей, поэтому время не зависит от того, сколько всего заказов у пользователя;
- `INCLUDE` не добавлен: `SELECT *` требует всех колонок, `Index Only Scan` здесь невозможен.

Глубокие страницы. Индекс не спасает от `OFFSET`: для `LIMIT 20` `OFFSET 20000` планировщик вернулся к `Bitmap + Sort (external merge)` — 30,1 мс. `Keyset-пагинация` (`AND created_at < <последнее значение предыдущей страницы>`) по тому же индексу даёт ту же страницу за **0,068 мс** и 23 буфера.

1.15 Задание 16. Index Only Scan

Листинг 13 – SQL-запрос (задание 16)

```
SELECT id, user_id FROM orders WHERE user_id = 123;
```

Таблица 14 – Задание 16. Index Only Scan

	До: (<code>user_id</code>)	После: (<code>user_id</code>) INCLUDE (<code>id, status, created at</code>)
user 123	Index Scan, 21 буфер, 0,082 мс	Index Only Scan , Heap Fetches: 0, 4 буфера, 0,106 мс
user 100001	Bitmap Heap Scan, 601 буфер, 7,1 мс	Index Only Scan , Heap Fetches: 0, 348 буферов, 8,2 мс
user 100001, <code>SELECT id, amount</code>	—	Bitmap Heap Scan (amount нет в индексе)
user 100001 после	—	Index Only Scan , Heap Fetches:

	До: (user_id)	После: (user_id) INCLUDE (id, status, created_at)
UPDATE 2 027 строк без VACUUM		51 125, 931 буфер, 11,9 мс
... после VACUUM	—	Index Only Scan, Heap Fetches: 0, 362 буфера, 7,1 мс

По времени Index Only Scan здесь не выиграл: все страницы таблицы уже в памяти, а 50 000 строк пользователя 100001 лежат компактно на 556 страницах. Выигрыш виден по числу прочитанных страниц, а на реальном диске и при холодном кэше он превращается и в выигрыш по времени.

1) **Отличие от Index Scan.** Index Scan находит в индексе адрес строки и затем читает саму строку из таблицы. Index Only Scan берёт все нужные колонки прямо из индекса и в таблицу не ходит.

2) **Почему можно не обращаться к таблице?** Все запрошенные колонки есть в индексе. Кроме того, visibility map отмечает страницу как all-visible, то есть все строки на ней видны всем транзакциям, и проверять видимость строки по MVCC не нужно.

3) **Необходимые условия:**

- все колонки из SELECT, WHERE и ORDER BY есть в индексе (в ключе или в INCLUDE);

- тип индекса это поддерживает (B-tree поддерживает);

- visibility map актуальна, то есть недавно проходил VACUUM.

Эксперимент с UPDATE показал: после изменения строк страницы теряют отметку all-visible, и каждая строка снова проверяется в таблице (Heap Fetches: 51 125);

- планировщик считает такой план дешевле.

INCLUDE. Колонки из INCLUDE хранятся только в листьях B-tree. Они не участвуют в поиске и сортировке, но позволяют Index Only Scan. Цена — размер: idx_orders_user_id_include занимает 57 МБ против 9,4 МБ у (user_id).

1.16 Задание 17. Partial Index

Листинг 14 – SQL-запрос (задание 17)

```
SELECT * FROM orders WHERE status = 'NEW' ORDER BY created_at; -- 10 409 строк
SELECT * FROM orders WHERE status = 'NEW' ORDER BY created_at LIMIT 20; -- типичный запрос
очереди
```

Таблица 15 – Задание 17. Partial Index

Вариант	Полная выборка	LIMIT 20	Размер индекса
только idx_orders_status	Bitmap + Sort — 11,3 мс	Index Scan idx_orders_created_at + Filter (отброшено 1 903) — 1,69 мс	7,2 МБ
обычный (status, created_at)	Bitmap + Sort — 12,3 мс	Index Scan, Index Cond: status = 'NEW' — 0,101 мс	39 МБ
частичный (created_at) WHERE status = 'NEW'	Bitmap + Sort — 11,6 мс	Index Scan idx_orders_new — 0,096 мс	248 кБ

Листинг 15 – План выполнения (задание 17)

```
Limit (actual time=0.048..0.077 rows=20 loops=1)
-> Index Scan using idx_orders_new on orders (actual time=0.048..0.075 rows=20 loops=1)
Execution Time: 0.096 ms
```

У частичного индекса в плане даже нет Index Cond: всё, что в нём лежит, по определению имеет статус NEW. Полную выборку (10 тысяч строк) ни один индекс от сортировки не избавил. Причина та же, что в задании 14: Bitmap плюс quicksort в памяти (1,2 МБ) планировщик оценивает дешевле.

1) **Преимущество partial index.** Та же скорость, что у составного индекса, при размере в **160 раз меньше** (248 кБ против 39 МБ): он содержит только 1 % строк. Строки с другими статусами его не затрагивают, так что запись дешевле, а сам индекс целиком помещается в кэш.

2) **Когда полезен.** Когда запросы постоянно фильтруют по одному и тому же редкому условию:

- очереди необработанных записей (status = 'NEW');
- «мягкое удаление» (WHERE deleted_at IS NULL);
- активные записи;
- уникальность внутри подмножества (частичный UNIQUE, например «одна активная подписка на пользователя»).

3) **Почему не один partial index на все запросы.** Индекс содержит только строки, удовлетворяющие предикату, и планировщик применяет его, лишь когда может доказать, что WHERE запроса влечёт этот предикат. Для `status = 'PAID'` он бесполезен: этот запрос пошёл через `idx_orders_created_at`. Частичный индекс под каждое значение — это тот же полный индекс, только хуже.

1.17 Задание 18. Expression Index

Таблица `users` — 500 001 строка, `email` в разном регистре (`User2@Example.com`, `user1@example.com`, `Test@Example.com`).

Таблица 16 – Задание 18. Expression Index

Запрос / индексы	План	Execution Time
<code>LOWER(email) = 'test@example.com', есть idx_users_email</code>	Parallel Seq Scan	62,4 мс
<code>email = 'Test@Example.com', тот же индекс</code>	Index Scan idx_users_email	0,059 мс
<code>LOWER(email) = 'test@example.com', есть idx_users_lower_email</code>	Index Scan idx_users_lower_email	0,060 мс

Почему обычный индекс по email не используется? Он хранит значения `email` как есть, отсортированными. Условие наложено на другое значение — `lower(email)`, и порядок `email` не совпадает с порядком `lower(email)`. Сопоставить условие с отрезком индекса невозможно, поэтому остаётся только вычислить `lower()` для каждой строки.

Вывод. Индекс применим, только если выражение в запросе в точности совпадает с проиндексированным (колонок или выражением). Индекс по выражению хранит уже вычисленное `lower(email)`, и запрос должен использовать то же выражение: `upper(email)` или `ILIKE` его не задействуют. Функция должна быть `IMMUTABLE`, а `ANALYZE` собирает статистику и по выражению. Альтернативы — хранить нормализованный `email` или использовать тип `citext`.

1.18 Задание 19. Цена индексов на запись

Одинаковые данные (`setseed(0.42)`) вставлялись в таблицу только с РК и в таблицу с РК и пятью индексами: `user_id`, `status`, `created_at`, `(user_id, created_at DESC)`, `(status, created_at)`.

Таблица 17 – Задание 19. Цена индексов на запись

Операция	Только РК	РК + 5 индексов	Во сколько раз
INSERT 1 000 000 строк	3,49 с	17,81 с	×5,1
UPDATE 100 000 строк (меняется <code>status</code>)	0,44 с	2,51 с	×5,8
DELETE 100 000 строк	0,09 с	0,13 с	×1,5
Размер индексов (таблица 93 МБ)	24 МБ	162 МБ	×6,8

1) **Как изменилось время INSERT?** Выросло в 5 раз.

2) **Почему?** На каждую строку теперь приходится обновить 6 B-tree вместо одного. Индексы по случайным ключам (`user_id`, `status` + `created_at`) вставляют записи в случайные страницы, что вызывает промахи кэша, расщепления страниц и дополнительные записи в WAL.

3) **Что происходит с индексами при добавлении строки?** Для каждого индекса вычисляется ключ, выполняется спуск по дереву до нужного листа, туда вставляется пара «ключ → адрес строки». Если лист заполнен, страница делится, и всё это пишется в WAL.

— UPDATE в PostgreSQL создаёт новую версию строки, поэтому записи добавляются во **все** индексы. Исключение — HOT-обновление, возможное, только если не меняется ни одна индексируемая колонка и на странице есть место.

— DELETE лишь помечает версию строки удалённой, индексы не трогает: их чистит позже VACUUM. Поэтому DELETE почти не замедлился, но работы у VACUUM прибавилось.

1.19 Задание 20. Неиспользуемые индексы

Листинг 16 – Статистика `pg_stat_user_indexes` к концу экспериментов

relname	indexrelname	idx_scan	size
-----+	-----+	-----+	-----

users	users_pkey	0	11 MB
orders	idx_orders_new	1	248 kB
users	idx_users_lower_email	1	19 MB
users	idx_users_email	1	19 MB
orders	orders_pkey	2	23 MB
orders	idx_orders_user_created_at_desc	3	32 MB
orders	idx_orders_created_at	3	23 MB
orders	idx_orders_user_id_include	5	57 MB
orders	idx_orders_user_id	11	9448 kB
orders	idx_orders_status	15	7232 kB

1) **Какие индексы использовались чаще всего?** `idx_orders_status` (15) и `idx_orders_user_id` (11): они дольше всех существуют и участвуют в большинстве экспериментов. `EXPLAIN ANALYZE` выполняет запрос, поэтому тоже увеличивает счётчик, а простой `EXPLAIN` — нет.

2) **Есть ли неиспользуемые?** `users_pkey` — `idx_scan = 0`: в задании 18 мы искали только по email.

3) **Можно ли считать индекс с `idx_scan = 0` бесполезным?** Нет:

- счётчик считается с момента последнего сброса статистики и может не охватывать редкие, но важные запросы: месячный отчёт, админку, миграции;

- статистика у каждого сервера своя: индекс может активно работать на реплике;

- РК и `UNIQUE` обеспечивают ограничение целостности независимо от того, читают ли их;

- индекс мог быть только что создан.

4) **Риски удаления индекса:**

- деградация редкого запроса до `Seq Scan` по большой таблице, вплоть до таймаутов;

- потеря ограничения уникальности;

- медленные каскадные удаления по внешним ключам;

- долгое пересоздание на большой таблице.

Безопаснее собрать статистику за длительный период на всех узлах, проверить `pg_stat_statements` и удалять через миграцию с откатом (`DROP INDEX CONCURRENTLY`).

1.20 Задание 21. Оптимизация запроса

Листинг 17 – SQL-запрос (задание 21)

```
SELECT id, amount, status, created_at
FROM orders
WHERE user_id = $1
  AND status = 'PAID'
  AND created_at >= NOW() - INTERVAL '30 days'
ORDER BY created_at DESC
LIMIT 50;
```

Запрос выполнялся как `PREPARE q21(BIGINT) + EXPLAIN (ANALYZE, BUFFERS) EXECUTE q21(...)`, то есть с настоящим параметром `$1`.

1–2. Исходный план (индексы `(user_id)`, `(status)`, `(created_at)`), пользователь 100001:

Листинг 18 – План выполнения (задание 21)

```
Limit (actual time=14.887..14.898 rows=50 loops=1)
  Buffers: shared hit=10 read=604
  -> Sort (Sort Key: created_at DESC, Sort Method: top-N heapsort)
    -> Bitmap Heap Scan on orders (actual rows=377 loops=1)
        Recheck Cond: ((user_id = '100001') AND (created_at >= ...) AND (status =
        'PAID'))
        Heap Blocks: exact=264
        -> BitmapAnd
            -> Bitmap Index Scan on idx_orders_user_id (actual rows=50000)
            -> Bitmap Index Scan on idx_orders_created_at (actual rows=43424)
            -> Bitmap Index Scan on idx_orders_status (actual rows=209307)
  Execution Time: 15.077 ms
```

Для пользователя 123 (в выборке 0 строк): `Limit` ← `Sort` ← `Index Scan idx_orders_user_id + Filter` по `status` и `created_at` — 0,217 мс.

3. Проблемы:

- чтобы найти 377 строк, три индекса читаются по своим условиям целиком — около 300 000 записей и 350 страниц;
- `Sort`: все 377 строк надо собрать и отсортировать, прежде чем вернуть первые 50;
- за каждой строкой идём в таблицу за `amount` (264 страницы);
- стоимость растёт вместе с активностью пользователя и размером таблицы.

4. Предлагаемый индекс:

Листинг 19 – SQL-запрос (задание 21)

```
CREATE INDEX idx_orders_user_status_created_at
ON orders(user_id, status, created_at DESC)
```

```
INCLUDE (id, amount);
```

5. Порядок колонок:

– `user_id` и `status` — условия равенства, поэтому идут первыми. После них все подходящие записи лежат одним отрезком. `user_id` поставлен первым, потому что он намного селективнее, а ещё такой индекс по левому префиксу обслуживает запросы только по `user_id`. Индекс, начинающийся со `status`, подошёл бы только для редких запросов по одному статусу;

– `created_at DESC` — последним: это одновременно диапазон и сортировка. Колонка после диапазона уже не была бы упорядочена, поэтому диапазонная колонка всегда ставится в конец;

– `INCLUDE (id, amount)` — оставшиеся колонки из `SELECT`, чтобы получить Index Only Scan.

6–8. Результат:

Листинг 20 – План выполнения (задание 21)

```
Limit (actual time=0.063..0.074 rows=50 loops=1)
  Buffers: shared hit=1 read=3
  -> Index Only Scan using idx_orders_user_status_created_at on orders
        Index Cond: ((user_id = '100001') AND (status = 'PAID') AND (created_at >= (now() - '30
days'::interval)))
        Heap Fetches: 0
Execution Time: 0.116 ms
```

Таблица 18 – Задание 21. Оптимизация запроса

Вариант индекса	user 123	user 100001	Buffers (100001)	План (100001)
До: (user_id), (status), (created_at)	0,217 мс	15,08 мс	614	BitmapAnd из трёх индексов + Sort
(user_id, created_at DESC)	—	0,604 мс	284	Index Scan, status фильтром (отброшено 230)
(user_id, status, created_at DESC) INCLUDE (amount)	0,073 мс	0,139 мс	53	Index Scan, всё в Index Cond, без Sort
Итог: ... INCLUDE (id, amount)	—	0,116 мс	4	Index Only Scan, Heap Fetches: 0

Время для активного пользователя сократилось в **130 раз** (15,08 → 0,116 мс), число прочитанных страниц — в 150 раз. У индекса есть цена: 66 МБ, больше любого другого индекса на таблице. С ним индекс (`user_id`) становится избыточным (левый префикс), и его можно удалить.

2 Оптимизация собственного сервиса TaskManager

Сервис — REST API на .NET 8 (ASP.NET Core, EF Core 8 + Npgsql), PostgreSQL 16, миграции Liquibase, всё запускается через docker compose up.

Таблица 19 – Данные сгенерированы скриптом scripts/generate-data.sql

Сущность	Строк
Пользователи	10 001
Проекты	1 000
Теги	50
Связи пользователь–проект	19 988
Задачи	1 000 000
Связи задача–тег	797 847

Задачи распределены по пользователям неравномерно: медиана — 72 задачи, а у gen_user_1 их 10 041 (1 % таблицы). Таблица tasks занимает 135 МБ, её индексы — 87 МБ.

2.1 Задание 22. Scaling Entity

1) **Таблица:** tasks.

2) **За что отвечает:** задача пользователя — название, описание, статус (Pending / InProgress / Completed / Cancelled), приоритет, срок, автор (user_id), проект (project_id), теги (М:М через task_tags), created_at / updated_at.

3) **Почему растёт быстрее остальных:** пользователи, проекты и теги — справочные сущности, их создают редко. Задачи создают постоянно, каждый пользователь, и после выполнения они не удаляются: история нужна для статистики. Число задач растёт как «пользователи × время», вместе с ним растёт task_tags.

4) **Ожидаемый объём через год:** при 10 000 активных пользователей и ~3 новых задачах в рабочий день — $10\,000 \times 3 \times 250 \approx 7,5$ млн строк в tasks. Это около 1 ГБ данных и ~0,6 ГБ индексов (сейчас 1 млн строк = 135 МБ + 87 МБ), плюс ~6 млн строк в task_tags.

5) Поля для поиска и фильтрации:

- `user_id` — каждый запрос обычного пользователя;
- `created_at` — сортировка всех списков;
- `status`, `priority`, `project_id`, `due_date` — фильтры `GET /api/tasks`;
- `id` — карточка задачи и `JOIN` с `task_tags`.

2.2 Задание 23. Реальные запросы

SQL снят из лога PostgreSQL. Скрипт `scripts/lab-01/part2-capture-sql.sh`:

- 1) на время включает `log_min_duration_statement = 0`;
- 2) вызывает API от имени активного пользователя (`gen_user_1`), типичного пользователя (`gen_user_5000`) и администратора (`gen_admin`);
- 3) возвращает настройку обратно.

Полный лог с параметрами — `docs/lab-01/results/part2-01-captured-sql.txt`. Каждый вызов `GET /api/tasks` выполняет два запроса: `count(*)` для пагинации и выборку страницы.

Листинг 21 – Query 1 — поиск по идентификатору (`GET /api/tasks`, общее число задач пользователя)

```
SELECT count(*)::int
FROM tasks AS t
WHERE t.user_id = $1
```

Листинг 22 – Query 2 — несколько условий (`GET /api/tasks?status=InProgress&priority=High`)

```
SELECT count(*)::int
FROM tasks AS t
WHERE t.user_id = $1 AND t.status = $2 AND t.priority = $3
```

Query 3 — фильтрация + сортировка + ограничение (`GET /api/tasks`, страница). EF Core сначала выбирает страницу задач, затем присоединяет проект и теги:

Листинг 23 – SQL-запрос (задание 23)

```
SELECT t0.id, t0.created_at, /* ... все колонки задачи, проекта и тега ... */ p.name
FROM (
    SELECT t.id, t.created_at, t.description, t.due_date, t.priority,
           t.project_id, t.status, t.title, t.updated_at, t.user_id
    FROM tasks AS t
    WHERE t.user_id = $1
    ORDER BY t.created_at DESC
    LIMIT $2 OFFSET $3
) AS t0
LEFT JOIN projects AS p ON t0.project_id = p.id
LEFT JOIN (
    SELECT t2.task_id, t2.tag_id, t3.id, t3.color, t3.created_at, t3.name
    FROM task_tags AS t2
    INNER JOIN tags AS t3 ON t2.tag_id = t3.id
) AS t1 ON t0.id = t1.task_id
ORDER BY t0.created_at DESC, t0.id, p.id, t1.task_id, t1.tag_id
```

Дополнительно исследованы варианты для администратора, который видит все задачи:

- **Query 3b** — та же страница без WHERE, плюс INNER JOIN users;
- **Query 3c** — то же с WHERE t.status = \$1;
- **Query 4** — GET /api/tasks/stats: SELECT t.status, count(*), count(*) FILTER (WHERE t.due_date < \$1 AND t.status IN (0, 1)) FROM tasks AS t GROUP BY t.status ORDER BY t.status;
- **Query 5** — SELECT count(*)::int FROM tasks AS t для пагинации администратора.

Все запросы воспроизводятся скриптом scripts/lab-01/part2-explain.sql через PREPARE / EXECUTE с plan_cache_mode = force_custom_plan. Так их выполняет и Npgsql: план строится с реальными значениями параметров.

2.3 Задание 24. Базовое измерение

EXPLAIN и EXPLAIN (ANALYZE, BUFFERS) каждого запроса сохранены в docs/lab-01/results/part2-02-explain-before.txt.

Таблица 20 – Задание 24. Базовое измерение

Запрос	Тип Scan (по tasks)	Execution Time	Buffers	Используемые индексы
Query 1, gen_user_1	Index Only Scan	1,08 мс	11	idx_tasks_user_id
Query 2, gen_user_1	Bitmap Heap Scan ← BitmapAnd из трёх индексов	12,79 мс	501	idx_tasks_user_id, idx_tasks_status, idx_tasks_priority

Запрос	Тип Scan (по tasks)	Execution Time	Buffers	Используемые индексы
Query 3, gen_user_1	Bitmap Heap Scan (10 041 строка) → Sort (top-N) → Limit 10	3,82 мс	248	idx_tasks_user_id; в JOIN — projects_pkey, pk_task_tags, tags_pkey
Query 3, gen_user_5000	Bitmap Heap Scan (68 строк) → Sort → Limit 10	0,52 мс	73	idx_tasks_user_id
Query 3, gen_user_1, OFFSET 5000	Bitmap Heap Scan → Sort (5 010 строк) → Limit	7,75 мс	255	idx_tasks_user_id
Query 3b, Admin	Parallel Seq Scan (1 000 000 строк) → Sort (top-N) → Limit 10	118,41 мс	17 451	для tasks — нет
Query 3c, Admin, Pending	Parallel Bitmap Heap Scan (200 659 строк) → Sort → Limit 10	71,26 мс	17 557	idx_tasks_status
Query 4, Admin	Parallel Seq Scan → HashAggregate	84,27 мс	17 296	—
Query 5, Admin	Parallel Index Only Scan	41,57 мс	790	idx_tasks_priority (самый маленький индекс таблицы)

Время ответа API (после прогрева EF Core, кэш Redis очищен) и длительность тех же SQL в БД по логу PostgreSQL. Время API — одиночные замеры с разбросом ± 10 мс, стабильная метрика — время SQL.

Таблица 21 – Задание 24. Базовое измерение

Endpoint	Пользователь	Время ответа API	В БД: count + страница
GET /api/tasks	gen_user_1	34 мс	0,5 + 3,6 мс
GET /api/tasks?status=InProgress &priority=High	gen_user_1	37 мс	12,2 + 12,3 мс
GET /api/tasks	gen_user_5000	12 мс	0,05 + 0,6 мс
GET /api/tasks	gen_admin	136 мс	23,9 + 99,2 мс
GET /api/tasks?status=Pending	gen_admin	89 мс	9,2 + 66,5 мс
GET /api/tasks/stats	gen_admin	76 мс	67,0 мс

2.4 Задание 25. Анализ текущих индексов

До изменений в БД было 29 индексов. idx_scan посчитан по семи вызовам API из задания 23, после pg_stat_reset(). Логины выполнялись до сброса, поэтому поиск по username в счётчики не попал.

Таблица 22 – Задание 25. Анализ текущих индексов

Индекс	Колонки	Какие запросы могут использовать	idx_scan	Размер	Вывод
--------	---------	----------------------------------	----------	--------	-------

Индекс	Колонки	Какие запросы могут использовать	idx_scan	Размер	Вывод
tasks_pkey	tasks(id)	GET/PUT/DELETE /api/tasks/{id}, FK из task_tags	0	38 МБ	нужен (эти endpoint'ы не вызывались)
idx_tasks_user_id	tasks(user_id)	все запросы пользователя, каскадное удаление пользователя	9	9,1 МБ	находит строки, но не даёт порядок created_at
idx_tasks_statuses	tasks(status)	фильтр status, BitmapAnd	6	6,3 МБ	4 значения, селективность низкая
idx_tasks_priority	tasks(priority)	фильтр priority, BitmapAnd, count(*) всей таблицы	6	6,3 МБ	4 значения, селективность низкая
idx_tasks_project_id	tasks(project_id)	фильтр projectId, FK ON DELETE SET NULL	0	9,3 МБ	нужен для фильтра и FK
idx_tasks_due_date	tasks(due_date)	фильтр dueDateFrom/To	0	19 МБ	нужен для фильтра (в выборке не вызывался)
pk_task_tags	task_tags(task_id, tag_id)	JOIN задач с тегами, уникальность	80	49 МБ	нужен
idx_task_tags_task_id	task_tags(task_id)	то же, что левый префикс РК	16	30 МБ	дубль префикса РК
idx_task_tags_tag_id	task_tags(tag_id)	каскадное удаление тега	0	5,1 МБ	нужен для FK
users_pkey / projects_pkey / tags_pkey	id	JOIN с задачами	48 / 76 / 84	< 0,5 МБ	нужны
users_email_key + idx_users_email	users(email)	логин по email, уникальность	0 / 0	0,8 + 0,8 МБ	дубль UNIQUE
users_username_key + idx_users_username	users(username)	логин, уникальность	0 / 0	0,5 + 0,5 МБ	дубль UNIQUE
tags_name_key + idx_tags_name	tags(name)	уникальность имени тега	0 / 0	16 + 16 кБ	дубль UNIQUE
api_keys_key_hash_key + idx_api_keys_key_hash	api_keys(key_hash)	аутентификация по API-ключу	—	мало	дубль UNIQUE
idempotency_keys_key + idx_idempotency_keys_key	idempotency_keys(key)	X-Idempotency-Key	—	мало	дубль UNIQUE
pk_user_projects	user_projects(user_id, project_id)	проверка участия в проекте, проекты пользователя	0	1,2 МБ	нужен
idx_user_projects_user_id	user_projects(user_id)	то же, что левый префикс РК	0	0,6 МБ	дубль префикса РК
idx_user_projects_project_id	user_projects(project_id)	участники проекта, каскадное удаление	0	0,2 МБ	нужен
idx_projects_name	projects(name)	поиск проекта по имени — в коде такого запроса нет	0	64 кБ	кандидат на удаление, пока оставлен
idx_idempotency_keys_expires_at	idempotency_keys(expires_at)	очистка просроченных	—	мало	нужен

Индекс	Колонки	Какие запросы могут использовать	idx_scan	Размер	Вывод
t		ключей			

Выводы:

– **7 индексов избыточны.** Пять — точные дубли UNIQUE-ограничений: Liquibase уже создаёт уникальный индекс для `unique="true"`, а changeset дополнительно создавал обычный на ту же колонку. Ещё два — левые префиксы составных первичных ключей. `idx_task_tags_task_id` (30 МБ) планировщик иногда выбирал вместо `pk_task_tags`, но это просто второе дерево по той же колонке. Все семь удаляются без потерь миграцией 010.

– `idx_tasks_status` и `idx_tasks_priority` построены по колонкам с 4 значениями. Для частых значений в одиночку они не нужны (задания 7–8), но участвуют в `BitmapAnd` и `count(*)`. Оставлены, пересмотреть по статистике `production`.

– `idx_tasks_due_date`, `idx_tasks_project_id`, `idx_task_tags_tag_id`, `idx_user_projects_project_id` в выборке не использовались, но обслуживают фильтры `dueDateFrom/To`, `projectId` и каскадные операции по внешним ключам. `idx_scan = 0` на семи запросах — не повод их удалять (задание 20).

– **Ни один индекс не отдаёт задачи в порядке `created_at`**, хотя этого требует каждый `GET /api/tasks`.

2.5 Задание 26. Потенциальные проблемы

Таблица 23 – Задание 26. Потенциальные проблемы

	Query 1: count по user_id	Query 2: user_id + status + priority	Query 3: страница пользователя	Query 3b / 3c: страница администратора
1. Проблема с производительностью	нет: 1 мс даже на 10 тыс. задач	да: 12,8 мс. Время растёт с размером всей таблицы: индексы по status и priority	да для активных пользователей: сортируются все их задачи ради 10 строк. Растёт с активностью пользователя, у	да, главная: 118 / 71 мс на страницу из 10 строк, растёт линейно с таблицей

	Query 1: count по user_id	Query 2: user_id + status + priority	Query 3: страница пользователя	Query 3b / 3c: страница администратора
		читаются на 200+ тыс. записей каждый	далёких страниц — ещё и с OFFSET	
2. Sequential Scan	нет	нет	нет	да: Parallel Seq Scan всех 1 млн строк (3b)
3. Селективность	1 % (gen_user_1), 0,007 % (типичный)	итог 0,04 % (414 строк), но status и priority по отдельности — 20–22 %	результат — 10 строк (LIMIT) из 68–10 041 строк пользователя	без фильтра (3b) или 20 % (3c), результат — 10 строк
4. Подходящий индекс	есть	составного нет, есть три одиночных	частично: (user_id) находит строки, но не даёт порядок	нет индекса по created_at
5. Можно улучшить индекс	не нужно	да: составной индекс с user_id первым	да: (user_id, created_at DESC) вместо (user_id)	да: (created_at DESC)
6. Можно изменить SQL	count (*) не обязательно считать при каждом запросе страницы	count (*) здесь дороже самой страницы	для далёких страниц — keyset-пагинация вместо OFFSET	точный count (*) всей таблицы (Query 5, 42 мс) заменить оценкой или кэшем

Query 4 (статистика администратора) по определению агрегирует всю таблицу. Seq Scan здесь — честный план, индекс не поможет. Варианты: кэшировать ответ в Redis, хранить счётчики отдельно, а при росте таблицы — партиционирование по created_at.

2.6 Задание 27. Выбранный запрос и инженерный процесс

Для оптимизации выбран главный endpoint сервиса GET /api/tasks: **Query 3** (страница пользователя) и **Query 3b/3c** (страница администратора). Это один и тот же LINQ-запрос в TaskRepository — `OrderByDescending(t => t.CreatedAt).Skip(...).Take(...)`.

1) **Проблема.** Список задач администратора отвечает 136 мс, из них 99 мс — выборка одной страницы из 10 задач. У активного пользователя страница в 7 раз дороже, чем у типичного.

2) **Измерение.** Время ответа API и длительность каждого SQL по логу PostgreSQL (задание 24).

3) EXPLAIN ANALYZE (Query 3b):

Листинг 24 – План выполнения (задание 27)

```
Limit (actual time=114.217..118.018 rows=10 loops=1)
-> Gather Merge (Workers Launched: 2)
    -> Sort (Sort Key: t.created_at DESC, Sort Method: top-N heapsort)
        -> Parallel Seq Scan on tasks t (actual rows=333333 loops=3)
            Buffers: shared hit=14710 read=2572
Execution Time: 118.413 ms
```

4) **Анализ плана.** Чтобы выдать 10 самых новых задач, PostgreSQL читает все 1 000 000 строк (17 тыс. страниц) и держит 10 лучших в куче. Параллельность лишь делит эту работу на три процесса. Для пользователя (Query 3) то же в меньшем масштабе: Bitmap Heap Scan собирает все его 10 041 задачу и сортирует. Время растёт линейно с таблицей и с активностью пользователя. При ожидаемых 7,5 млн строк это около секунды на одну страницу администратора.

5) **Выбор индекса.** Сортировка по `created_at DESC` плюс `LIMIT` — как раз тот случай, когда B-tree заменяет сортировку и позволяет остановиться после N записей (задания 14–15):

- `(user_id, created_at DESC)` — для пользователя. `user_id` — равенство, поэтому идёт первым; внутри одного пользователя записи уже упорядочены как в `ORDER BY`;

- `(created_at DESC)` — для администратора: список без фильтра читается с начала индекса. Фильтр по `status` (Query 3с) проверяется на лету: при статусе в 5–20 % таблицы 10 подходящих строк находятся через десятки записей, поэтому отдельный `(status, created_at)` не нужен;

- `idx_tasks_user_id (user_id)` удаляется: его полностью заменяет левый префикс нового составного индекса (задание 12), в том числе для `count (*)` и проверок внешнего ключа;

- `INCLUDE` не нужен: EF выбирает все колонки задачи, `Index Only Scan` невозможен.

6) **Создание индекса** — миграция (задание 28).

7) **Повторное измерение и сравнение** — задание 29.

2.7 Задание 28. Изменение в проекте

Индексы добавлены миграцией Liquibase
docker/liquibase/changesets/009-create-tasks-created-at-
indexes.xml и подключены в db.changelog-master.xml:

Листинг 25 – Фрагмент миграции Liquibase (задание 28)

```
<changeSet id="009-create-tasks-created-at-indexes" author="taskmanager"
runInTransaction="false">
  <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_user_id_created_at ON tasks
(user_id, created_at DESC)</sql>
  <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_created_at ON tasks (created_at
DESC)</sql>
  <rollback>...</rollback>
</changeSet>

<changeSet id="009-drop-redundant-idx-tasks-user-id" author="taskmanager"
runInTransaction="false">
  <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_user_id</sql>
  <rollback>...</rollback>
</changeSet>
```

– CONCURRENTLY строит индекс, не блокируя запись в tasks, — на растущей таблице это обязательно. Такой CREATE INDEX нельзя выполнять внутри транзакции, поэтому стоит runInTransaction="false".

– IF NOT EXISTS / IF EXISTS делают changeSet безопасным при повторном запуске, у каждого changeSet есть rollback.

– По итогам задания 25 отдельная миграция 010-drop-duplicate-indexes.xml удаляет 7 дублирующих индексов.

– EF-модель (TaskItemConfiguration, TaskTagConfiguration, UserProjectConfiguration) приведена в соответствие со схемой.

Воспроизводимость. Миграции применяет сервис liquibase при каждом docker compose up, а сервис api стартует только после его успешного завершения. Проверено двумя способами:

– на существующей БД: Run: 3, Previously run: 8 (docs/lab-01/results/part2-03-migration.txt);

– на пустой БД: Run: 11, Previously run: 0. В tasks созданы именно idx_tasks_user_id_created_at и idx_tasks_created_at, а idx_tasks_user_id отсутствует.

2.8 Задание 29. Повторное измерение

Результаты — docs/lab-01/results/part2-04-explain-after.txt и part2-05-captured-after.txt.

Query 3b — страница всех задач (Admin):

Листинг 26 – План выполнения (задание 29)

```
Limit (actual time=0.020..0.034 rows=10 loops=1)
  Buffers: shared hit=13
  -> Index Scan using idx_tasks_created_at on tasks t (actual time=0.020..0.033 rows=10
      loops=1)
Execution Time: 0.548 ms          -- весь запрос вместе с JOIN projects, users, task_tags, tags
```

Таблица 24 – Задание 29. Повторное измерение

Метрика	До	После
Тип Scan	Parallel Seq Scan + Gather Merge + Sort (top-N)	Index Scan + Limit, сортировки tasks нет
Execution Time	118,41 мс	0,548 мс (в 216 раз быстрее)
Actual Rows (прочитано строк tasks)	1 000 000 (333 333 × 3 процесса)	10
Buffers	17 451	103
Используемый индекс	нет	idx_tasks_created_at

Query 3 — страница активного пользователя gen_user_1:

Листинг 27 – План выполнения (задание 29)

```
Limit (actual time=0.016..0.027 rows=10 loops=1)
  -> Index Scan using idx_tasks_user_id_created_at on tasks t (actual rows=10
      loops=1)
      Index Cond: (user_id = '335fc496-f143-433c-b8b5-48277f10419e'::uuid)
      Buffers: shared hit=13
Execution Time: 0.353 ms
```

Таблица 25 – Задание 29. Повторное измерение

Метрика	До	После
Тип Scan	Bitmap Heap Scan + Sort (top-N) + Limit	Index Scan + Limit
Execution Time	3,82 мс	0,353 мс (в 11 раз быстрее)
Actual Rows (прочитано строк tasks)	10 041	10
Buffers	248	69
Используемый индекс	idx_tasks_user_id	idx_tasks_user_id_created_at

Все исследованные запросы:

Таблица 26 – Задание 29. Повторное измерение

Запрос	До	После	Итог
Query 1, count по user_id	1,08 мс, IOS idx_tasks_user_id, 11 buf	1,32 мс, IOS idx_tasks_user_id_created_at, 54 buf	чуть медленнее: новый индекс шире
Query 2, count user_id + status + priority	12,79 мс, BitmapAnd	11,56 мс, BitmapAnd	без изменений, проблема осталась
Query 3, gen user 1	3,82 мс	0,35 мс	в 11 раз быстрее
Query 3, gen user 5000	0,52 мс	0,41 мс	в пределах шума
Query 3, gen user 1, OFFSET 5000	7,75 мс (Sort)	2,59 мс (Index Scan по 5010 записям, 5 078 buf)	в 3 раза быстрее, но растёт с OFFSET
Query 3b, Admin	118,41 мс	0,55 мс	в 216 раз быстрее
Query 3c, Admin, Pending	71,26 мс	0,45 мс (Rows Removed by Filter: 29)	в 159 раз быстрее
Query 4, статистика	84,27 мс	87,27 мс	без изменений (агрегат всей таблицы)
Query 5, count всех задач	41,57 мс	39,60 мс	без изменений

Таблица 27 – Время ответа API (после прогрева, кэш Redis очищен; в скобках — время SQL по логу PostgreSQL)

Endpoint	До	После
GET /api/tasks — gen_admin	136 мс (count 23,9 + страница 99,2)	34 мс (count 22,0 + страница 0,45)
GET /api/tasks?status=Pending — gen_admin	89 мс (9,2 + 66,5)	23 мс (9,3 + 0,41)
GET /api/tasks — gen_user_1	34 мс (0,5 + 3,6)	15 мс (0,8 + 0,45)
GET /api/tasks?status=InProgress&priority=High — gen_user_1	37 мс (12,2 + 12,3)	40 мс (12,5 + 0,46)
GET /api/tasks — gen_user_5000	12 мс	12 мс
GET /api/tasks/stats — gen_admin	76 мс	74 мс

Цена изменения:

- на tasks: +39 МБ (idx_tasks_user_id_created_at) +21 МБ (idx_tasks_created_at) –9 МБ (idx_tasks_user_id);
- миграция 010 освободила 32 МБ, из них 30 МБ — idx_task_tags_task_id;
- вставка задачи теперь обновляет 7 индексов вместо 6, а вставка связи задача–тег — 2 вместо 3;

– всего индексов в БД — 23 вместо 29.

Что осталось (следующие шаги за рамками этой работы):

– `count(*)` для пагинации теперь дороже самой страницы: 22 мс у администратора и 12 мс при фильтре `status + priority`. Варианты: не считать точное количество для больших выборок (оценка через `pg_class.reltuples`), кэшировать его или перейти на `keyset`-пагинацию без общего числа страниц;

– Query 2 (`user_id + status + priority`) по-прежнему собирает `BitmapAnd` из трёх индексов. Если этот фильтр популярен, нужен составной индекс, начинающийся с `user_id`, — после проверки его цены на запись;

– GET `/api/tasks/stats` для администратора (~85 мс) — агрегат всей таблицы. Решается кэшированием или партиционированием, а не индексом.

3 Почему нельзя индексировать всё (задание 30)

Индекс ускоряет чтение, за которое заплачено записью и местом. Индекс на каждой колонке — это плата без гарантии пользы.

- **Размер индексов и стоимость хранения.** Каждый индекс — отдельная структура на диске и в памяти. В задании 19 пять индексов заняли 162 МБ при таблице 93 МБ, один покрывающий индекс из задания 21 — 66 МБ. Индексы вытесняют из `shared_buffers` полезные страницы таблицы, увеличивают бэкапы, объём WAL и время восстановления, и всё это копируется на каждую реплику.

- **INSERT.** Каждая новая строка вставляется во все индексы: с пятью индексами вставка миллиона строк замедлилась в 5,1 раза ($3,5 \rightarrow 17,8$ с).

- **UPDATE.** Из-за MVCC новая версия строки получает записи во всех индексах, а изменение индексируемой колонки к тому же отключает HOT-обновления. Замедление — в 5,8 раза.

- **DELETE.** Сам по себе почти не дорожает ($\times 1,5$), но мёртвые записи остаются в каждом индексе, и работы у `VACUUM` становится пропорционально больше.

- **Необходимость обновления индексов.** Индексы разрастаются (bloat) и требуют `VACUUM`, а иногда `REINDEX`. Создание индекса на большой таблице занимает время и требует `CREATE INDEX CONCURRENTLY`, чтобы не блокировать запись.

- **Влияние на скорость записи.** Для сервисов с активной записью (задачи, заказы, события) именно запись — узкое место, и каждый лишний индекс снижает пропускную способность.

- **Реальное использование запросами.** Индекс на отдельной колонке часто вообще не применим:

- при низкой селективности планировщик выбирает Seq Scan (`status = 'DELIVERED', 75 %`);

- условие на вторую колонку составного индекса его не задействует (задание 13);
- выражение вместо колонки не совпадает с индексом (`lower(email)`, задание 18);
- сортировка требует составного индекса, а не двух одиночных (задание 14).

А нужные запросам составные, частичные и покрывающие индексы механическое «индексирование всех колонок» не создаёт.

Индексы нужно проектировать под конкретные запросы: измерить, выбрать, проверить по `EXPLAIN ANALYZE`, а затем периодически пересматривать по `pg_stat_user_indexes`, удаляя избыточные.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены способы доступа к данным в PostgreSQL и проведены измерения на тестовой таблице из 1 000 000 строк и на базе данных собственного сервиса.

По результатам первой части установлено:

- индекс ускоряет выборку по селективному условию на два порядка: поиск заказов пользователя — с 32,7 мс до 0,12–0,27 мс;
- при выборке большой доли таблицы (75 % строк по статусу, 50 % по дате) планировщик выбирает Seq Scan, и это оптимальный план;
- в составном индексе колонки с условием равенства должны стоять перед колонками диапазона и сортировки; индекс (`user_id, created_at DESC`) устраняет операцию Sort и позволяет остановиться после LIMIT записей;
- частичный индекс даёт ту же скорость, что и составной, при размере в 160 раз меньше (248 кБ против 39 МБ);
- индекс по выражению применим только к запросу с тем же выражением;
- пять дополнительных индексов замедляют вставку в 5,1 раза, а обновление индексируемой колонки — в 5,8 раза;
- итоговый запрос задания 21 ускорен в 130 раз (с 15,08 мс до 0,116 мс) за счёт индекса (`user_id, status, created_at DESC`) INCLUDE (`id, amount`).

Во второй части работы из журнала PostgreSQL получен реальный SQL, который формирует EF Core для сервиса TaskManager (1 000 000 задач). Главной проблемой оказался список задач GET /api/tasks: сортировка по `created_at` выполнялась без индекса, и чтобы выдать администратору 10 строк, PostgreSQL читал всю таблицу (118 мс). Миграцией Liquibase добавлены индексы (`user_id, created_at DESC`) и (`created_at DESC`), избыточный индекс по `user_id` и семь индексов-дубликатов удалены. В

результате страница администратора выполняется за 0,55 мс вместо 118 мс, страница активного пользователя — за 0,35 мс вместо 3,8 мс, а время ответа API для администратора сократилось со 136 мс до 34 мс.

Оставшиеся узкие места — подсчёт общего числа записей для пагинации и агрегирующий запрос статистики — индексами не решаются и требуют изменения запросов или кэширования. Главный вывод работы: индексы нужно проектировать под конкретные запросы и подтверждать каждое изменение измерением, а неиспользуемые и дублирующие индексы — удалять, поскольку каждый индекс замедляет запись.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 PostgreSQL 16 Documentation. Chapter 11. Indexes [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/indexes.html> (дата обращения: 10.09.2026).

2 PostgreSQL 16 Documentation. 14.1. Using EXPLAIN [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/using-explain.html> (дата обращения: 10.09.2026).

3 PostgreSQL 16 Documentation. 11.9. Index-Only Scans and Covering Indexes [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/indexes-index-only-scans.html> (дата обращения: 10.09.2026).

4 PostgreSQL 16 Documentation. CREATE INDEX [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/sql-createindex.html> (дата обращения: 10.09.2026).

5 Liquibase Documentation. Change Types: sql [Электронный ресурс]. – URL: <https://docs.liquibase.com/change-types/sql.html> (дата обращения: 10.09.2026).

6 Лабораторная работа № 1. Индексы и EXPLAIN ANALYZE в PostgreSQL : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 20 с.

7 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017.

ПРИЛОЖЕНИЕ А

Миграция с индексами для GET /api/tasks

Миграция Liquibase, добавленная в проект по результатам заданий 27–29. Применяется автоматически при запуске `docker compose up`.

Листинг А.1 – 009-create-tasks-created-at-indexes.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
  xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
    http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.25.xsd">

  <!--
    JP №1 (docs/lab-01-indexes.md, задания 27-29).
    GET /api/tasks всегда сортирует по created_at DESC и берёт одну страницу
    (LIMIT/OFFSET):
      - обычный пользователь: WHERE user_id = ? ORDER BY created_at DESC -> (user_id,
        created_at DESC)
      - Admin / API Key:      ORDER BY created_at DESC [WHERE status = ?] -> (created_at
        DESC)
    Без этих индексов PostgreSQL читает все строки пользователя (или всю таблицу) и
    сортирует их.

    CONCURRENTLY не блокирует запись в tasks во время построения индекса,
    поэтому changeSet выполняется вне транзакции.
  -->
  <changeSet id="009-create-tasks-created-at-indexes" author="taskmanager"
    runInTransaction="false">
    <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_user_id_created_at ON tasks
      (user_id, created_at DESC)</sql>
    <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_created_at ON tasks (created_at
      DESC)</sql>

    <rollback>
      <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_created_at</sql>
      <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_user_id_created_at</sql>
    </rollback>
  </changeSet>

  <!--
    idx_tasks_user_id (user_id) стал избыточным: любой поиск по user_id (в том числе
    проверка внешнего ключа при каскадном удалении пользователя) обслуживает левый
    префикс idx_tasks_user_id_created_at. Лишний индекс только замедляет запись.
  -->
  <changeSet id="009-drop-redundant-idx-tasks-user-id" author="taskmanager"
    runInTransaction="false">
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tasks_user_id</sql>

    <rollback>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_user_id ON tasks
        (user_id)</sql>
    </rollback>
  </changeSet>

</databaseChangeLog>
```

ПРИЛОЖЕНИЕ Б

Миграция удаления дублирующих индексов

Миграция Liquibase, добавленная по результатам анализа индексов в задании 25.

Листинг Б.1 – 010-drop-duplicate-indexes.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
  xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
    http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.25.xsd">

  <!--
    ЛР №1 (docs/lab-01-indexes.md, задание 25): индексы, которые полностью дублируют
    уже существующие. Поиск по этим колонкам продолжают обслуживать UNIQUE-ограничения
    и первичные ключи, а каждая вставка перестаёт обновлять лишнюю структуру.
  -->
  <changeSet id="010-drop-duplicate-indexes" author="taskmanager" runInTransaction="false">
    <!-- те же колонки, что у UNIQUE-ограничений -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_users_email</sql> <!-- =
users_email_key -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_users_username</sql> <!-- =
users_username_key -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_tags_name</sql> <!-- =
tags_name_key -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_api_keys_key_hash</sql> <!-- =
api_keys_key_hash_key -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_idempotency_keys_key</sql> <!-- =
idempotency_keys_key_key -->

    <!-- левый префикс составного первичного ключа -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_task_tags_task_id</sql> <!--
pk_task_tags (task_id, tag_id) -->
    <sql>DROP INDEX CONCURRENTLY IF EXISTS idx_user_projects_user_id</sql> <!--
pk_user_projects (user_id, project_id) -->

    <rollback>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_user_projects_user_id ON
user_projects (user_id)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_task_tags_task_id ON task_tags
(task_id)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_idempotency_keys_key ON
idempotency_keys (key)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_api_keys_key_hash ON api_keys
(key_hash)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tags_name ON tags (name)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_username ON users
(username)</sql>
      <sql>CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_users_email ON users (email)</sql>
    </rollback>
  </changeSet>

</databaseChangeLog>
```